Web Programming Project Report

Project 1: Statistics portal

1. **The introduction of the project**

   The project is a interactive visualization web page that based on “Tilastokeskus” for presenting data. Users could make requires on the Finland map for getting data concerning multiple statistical indicators of multiple municipalities. What’s more, users also could check the change of each data over years in the form of a line chart after clicking different towns and combine different data by themselves to form new indicators. And export the result into a picture. Data are fetched live from the Statistics Finland PxWeb API using three datasets: municipal elections (kvaa/152l.px), employment statistics (tyokay/115x.px) and population structure (vaerak/11ra.px). Municipality boundaries come from geo.stat.fi. The map and panel are side by side on desktop and stacked on mobile. The application was tested in Chrome, Edge, Firefox and Safari.

   All project used native HTML, CSS and JavaScript. No React or other heavy frameworks are adopted, and no build tools are used.

2. **Which tools I used**

   Leaflet: It plays an important role. It is used for the interactive map, GeoJSON layers.

   EChart: Generate the line charts and bar charts

   Html2canvas: Export the map into a picture

   Native JavaScript: used fetch, async and await for designing functions

   VS code : For developing this project

   Git/GitHub: Management of the project version

3. **Points Explanation**

   I am requesting 37 points. This project is the largest project I have completed so far. I spent nearly 2 weeks on it. Although many bug popping up during development, I tried my best effort to learn how build and fix it. Fortunately, I am happy with the final result. Despite frequent bugs occurs, I enjoyed the whole process of learning and fixing. Therefore, I believe my effort deserve the grade I aiming for.

4. **User manual**

a. Top Toolbar: It includes three operations: switch the data layer of the map, select a year from 2000 to 2024, and export the current map as a PNG image.

b. Data Shelf: There are 7 data cards in total. On desktop: drag a card onto the map to switch the displayed data layer. On touch screens: tap a card to switch the layer. The currently active layer is highlighted in blue.

c. Map Interaction: Hover over a municipality to see its name and the current indicator value. Click a municipality, and the right panel will show its full-time series data and all indicator values.

d. Formula Builder: There are three controls on the right of the toolbar: data item A, operator, and data item B. You can combine two indicators for calculation.

5. AI Declaration

1) Which AI I used: DeepSeek 4.1flash & ChatGPT

2)How and Where I used: DeepSeek and ChatGPT used as a teacher during the development. The points below are the places where I couldn't work out the solution by myself.

- Party colours (script.js, PARTY_COLORS). I did not know which colour is conventionally used for each Finnish political party. I asked the AI, which described the colours commonly used for these parties in Finnish media, and I entered them into the table.
- Class breaks (script.js, indicators). I did not know which numerical values to use as the six class breaks of each layer. The AI calculated them from the real distribution of the data, and explained why the unemployment rate and the dependency ratio need the colour order reversed compared with the employment rate.
- Data decoding (data.js, decodeJsonStat). The PxWeb API returns all values as one long one-dimensional array. I could not work out how to convert a set of dimension values into the correct position in that array. The AI derived and explained the index calculation.
- Selecting the winning party (data.js, loadElectionData). I could not write the loop that compares the share of every party inside one municipality and keeps the largest one. The AI explained the approach.
- Derived-indicator formula (script.js, buildCustomLayer). I was tutored through the logic of this feature: computing one new value per municipality, and then deriving the class breaks from the minimum and maximum of those values.
- Refactoring repeated code (data.js, pick). The AI suggested extracting the repeated "read one indicator for every year" loop into a single helper function instead of writing the same loop three times.
- Drag and drop (script.js, setupCard; style.css, .card). I had not used the HTML5 drag-and-drop API before. The AI explained that preventDefault() is required on dragover.
- Exporting an image (script.js, exportPng). The AI introduced the approach of using html2canvas together with toDataURL() and a download link.
- Chart initialisation (charts.js). The AI explained why the ECharts object has to be created only once and then updated with setOption(), instead of being created again on every redraw.
- Tooltip and popup content (script.js, setupTown). The AI explained that the content of a tooltip can be given as a function, so that the text is calculated each time it is needed and does not have to be re-bound after the data changes.
- Debugging. Several errors occurred that I could not locate on my own and that were found with AI help: a misspelled variable name, Object.keys written as Objects.keys, a function defined under a different name from the one used to call it, getElementById mistyped, and a function called before the data it needed had been merged.